

LineFollowerPipeline

Pure-Java, zero-dependency line-following vision processor

Single-file library • Java 8+ • No external dependencies • Updated October 2026

What's new in this version

- **Any two colours.** Line and floor colours are learned during calibration — white on black, black on blue, white or black on brown, even red on green. Shadows across the floor are handled.
- **Colour input.** New `calibrateYuv420` / `processFrameYuv420` (Android camera) and `calibrateArgb` / `processFrameArgb`, alongside grayscale.
- **Smooth slowdown.** Tracking now ramps down linearly with the remaining visible road before a dead end or a sharp corner, instead of dropping from 100 to 2 in one frame.
- **Steering recovers after turns.** When re-acquiring the line the pipeline prefers the band at the calibrated centre with the calibrated line width, and corrects a lock that ended up on a reflection or a neighbouring tape.
- **Fewer false detections.** A plain floor no longer "finds" a line in sensor noise (`minContrast`).
- **Richer callback.** `FrameData` adds calibration phase / progress / problem, learned colours, threshold, real line position and width, frame size.

What This Library Does

`LineFollowerPipeline.java` is a pure-Java, zero-dependency line-following vision processor. You feed it raw camera frames — **grayscale, YUV 4:2:0 or ARGB** — and it returns two integer control values your robot can act on directly: a **steering direction** and a **speed/tracking signal**. No OpenCV, no Android APIs, and no native libraries are required.

Requirements

- Java 8 or higher
- One file only: `LineFollowerPipeline.java`
- No external dependencies

Minimal Integration (4 Steps)

- 1 Create one instance (keep it alive for the session)

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();
```

- 2 Calibrate — feed frames until it returns `true`

```
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    grayscaleBytes = camera.getGrayscaleFrame();
}
```

- 3 Call `processFrame()` on every camera frame

```
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

- 4 Read the two output values

```
int steering = result[0]; // use for direction control
int tracking = result[1]; // use for speed control
```

Have colour frames? Use `calibrateYuv420(y, u, v, w, h) / processFrameYuv420(...)` (Android camera) or `calibrateArgb(...)` / `processFrameArgb(...)` instead — colour input separates even line/floor colours of the same brightness. See *Input Formats*.

Calibration is required. `processFrame()` must not be called until `calibrate()` has returned `true`. The robot must be correctly placed on the line — with floor visible on both sides — while calibrating. See *Calibration* below.

Input Formats NEW

Each format has a matching `calibrate` / `process` pair. Use the **same format** for both.

FORMAT	CALIBRATE	PROCESS	DATA
Grayscale	<code>calibrate(gray, w, h)</code>	<code>processFrame(gray, w, h)</code>	<code>byte[] gray</code> — $w \times h$ luma bytes
YUV 4:2:0 <i>recommended on Android</i>	<code>calibrateYuv420(y, u, v, w, h)</code>	<code>processFrameYuv420(y, u, v, w, h)</code>	<code>y</code> — $w \times h$ bytes; <code>u</code> , <code>v</code> — $((w+1)/2) \times ((h+1)/2)$ bytes each
ARGB	<code>calibrateArgb(argb, w, h)</code>	<code>processFrameArgb(argb, w, h)</code>	<code>int[] argb</code> — $w \times h$ packed 0xAARRGGBB

Important: all arrays must be packed with **no row padding** (and the U/V planes with no pixel interleaving). Android CameraX planes are padded / interleaved — strip that before passing the data in (see `copyLumaPlane` / `copyChromaPlane` in RoboPhoneApp's `MainActivity`). Arrays that are too short throw `IllegalArgumentException`.

Which format should I use?

- **Colour (YUV / ARGB)** separates any two colours that look different — even two colours of the **same brightness** (e.g. red on green).
- **Grayscale** works for any two colours that differ in **brightness** (white on black, black on blue, white on brown, black on white). Calibration reports `ISSUE_LOW_CONTRAST` and keeps retrying if the two colours have the same brightness.

Output Format

`processFrame()` returns a **reused** `int[2]` array. Copy the values immediately if you need to keep them — the same array is overwritten on the next call.

INDEX	NAME	RANGE	MEANING
<code>result[0]</code>	Steering	-100 ... +100	0 = line is centred. Negative = line is to the left, turn left. Positive = line is to the right, turn right.
<code>result[1]</code>	Tracking	2 ... 100	100 = straight path ahead, full speed. Drops as a turn or curve is detected, and ramps down linearly as the visible road ahead gets shorter (dead end, sharp corner). Never falls below <code>Config.trackingMinValue</code> (default 2), so the robot is never sent a full stop. Set that field to 0 for the full 0–100 range.

Typical robot wiring:

```
robot.setSteeringAngle(result[0]); // direction
robot.setSpeed(result[1]);        // speed – slows before turns and dead ends
```

Optional Configuration

To tune the pipeline for your camera mount, track, or robot body size, create a `Config` object and pass it **before the first frame**:

```
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
// --- apply your settings here ---
pipeline.configure(cfg); // must be called BEFORE the first frame
```

Line Colour

FIELD	TYPE	DEFAULT	DESCRIPTION
autoColor NEW	boolean	true	Learn the line and floor colours during calibration — any two different colours work and trackIsBright is ignored. false = classic grayscale mode using trackIsBright.
trackIsBright	boolean	true	Only used when autoColor = false. true = bright line on dark floor; false = dark line on bright floor.
colorCalibFrames NEW	int	5	Frames sampled to learn the colours.
colorMinSeparation NEW	float	4.0f	Minimum line-vs-floor colour separation (in standard deviations of the colour noise). Below this, calibration discards the samples and retries.
colorMinDistance NEW	float	12f	Minimum distance between the two learned colours (YUV grey-levels). Rejects "two colours" that are only sensor noise.
colorMinThreshold NEW	int	112	In colour mode the floor maps to 64 and the line to 192 internally; the threshold never drops below this, so noisy floor pixels are never taken for the line.
thresholdMargin	int	0	Bias added to the auto threshold. Increase if false positives occur; decrease if the line is being missed.
minContrast NEW	int	24	If the two halves of the Otsu split differ by less than this many grey-levels, the region is a plain surface with no line (prevents "finding" a line in sensor noise). 0 disables.
bottomIgnoreFrac	float	0.15f	Fraction of the frame bottom to ignore — excludes the robot's own body from the image. Increase until the chassis disappears from the scan.

Scan Strip (H-ROI — Near Line / Steering)

FIELD	TYPE	DEFAULT	DESCRIPTION
hRoiCenterYFrac	float	0.85f	Vertical position of the near scan strip as a fraction of frame height. 0.85 = 85 % down from the top.
hRoiThickPx	int	24	Height of the scan strip in pixels.

Look-Ahead Column (V-ROI — Turn Preview / Tracking)

FIELD	TYPE	DEFAULT	DESCRIPTION
vRoiHalfWidthPx	int	80	Half-width in pixels of the look-ahead column. Increase on tight curves so the line stays inside the scan window.
vRoiHeightFrac	float	0.5f	How far ahead to look, as a fraction of frame height. Larger = more warning of far turns, but noisier.
vRoiAdaptiveThreshold	boolean	true	Gives the look-ahead its own Otsu threshold. Recommended — the far line is dimmer than the near line.

Camera Mount

FIELD	TYPE	DEFAULT	DESCRIPTION
cameraTiltDeg	float	45f	How far the camera points below horizontal. Used for the look-ahead perspective taper, the look-ahead zero reference, and the road-length slowdown.

verticalFovDeg	float	60f	Vertical field of view of the camera. Most phone back cameras are ~55–65°.
----------------	-------	-----	--

Track Memory

FIELD	TYPE	DEFAULT	DESCRIPTION
trackMemory	boolean	true	Lock onto the line and only search a gate around its last position, so reflections elsewhere are ignored. false = search the whole width every frame.
trackMemoryGateMult	float	3.0f	Gate half-width as a multiple of the measured line width.
trackMemoryMinGatePx	int	48	Minimum gate width in pixels regardless of line width.
trackLostGraceFrames	int	8	Frames to wait after losing the line before searching the full frame for a new one. At 30 fps, 8 ≈ 0.25 s.
trackRelockFrames NEW	int	6	Lock correction. If a band that looks much more like the line (near the calibrated straight-ahead column, with the calibrated line width) is seen outside the lock for this many frames in a row, the lock moves to it — so a reflection or neighbouring tape grabbed after a turn cannot keep steering offset. 0 disables.

When searching for the line (start-up, or after it was lost), the pipeline no longer simply takes the biggest band in view: it prefers the band nearest the calibrated straight-ahead column whose width matches the line width measured during calibration. While the locked line is lost, steering reads exactly ± 100 toward the side it left.

Sensitivity & Tracking

FIELD	TYPE	DEFAULT	DESCRIPTION
minWeightFrac	float	0.01f	Minimum fraction of scan-strip pixels that must be "track" to trust the detection. Raise to reject weak signals.
trackOffsetFullFrac	float	1.0f	How far the look-ahead line must lean (fraction of half-frame width) to push Tracking to its minimum. Lower (e.g. 0.6) makes turns read earlier.
trackLengthSlowdown NEW	boolean	true	Also limit Tracking by how much road is still visible ahead (floor distance, perspective-corrected). Approaching a dead end or a sharp corner, Tracking ramps down linearly with the remaining road instead of dropping from 100 to the minimum in one frame. A fully visible straight road still reads 100.
trackLengthFullFrac NEW	float	0.8f	Fraction of the look-ahead distance the line must reach for the road-length part to read 100. Keeps the dim far edge from lowering Tracking on straight roads.
trackingMinValue	int	2	Floor for the Tracking output — <code>result[1]</code> never goes below it, so the robot never receives a full-stop signal. 0 allows the full 0–100 range.

Tracking Minimum Value

By default Tracking **never reaches zero**: `result[1]` is clamped to a floor of 2, so the robot always receives at least a small speed signal. This prevents mechanical problems caused by a complete stop command.

VALUE	EFFECT
2 (default)	The robot always gets at least a tiny speed signal.
3	Raise the floor further.
0	Disable the floor — allow the full 0–100 range.

Calibration Frames

FIELD	TYPE	DEFAULT	DESCRIPTION
calibFrames	int	10	Number of solid detections collected before locking the straight-ahead zero reference (their median) and the typical line width. Assumes the robot is correctly placed on the line.

Calibration

Before calling `processFrame()` you must calibrate. Place the robot correctly on the line, then call `calibrate()` (or the YUV / ARGB version) once per camera frame until it returns `true`:

```
// Phase 1 – Calibration (at startup, or when the user presses calibrate)
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    grayscaleBytes = camera.getGrayscaleFrame(); // keep feeding frames
}
// Phase 2 – Normal operation
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

Calibration has two phases:

1. **Learn the colours** NEW (`colorCalibFrames`, default 5 frames). The scan strip is sampled and split into two colour clusters (k-means). The **floor** is the cluster that fills both the left and right frame edges; the other is the **line**. If that fails — typically a shadow or uneven light making part of the floor almost as dark as a dark line — 3 and then 4 clusters are tried: the line is the cluster that forms one band and never reaches the frame edges, and all other clusters (lit floor, shadowed floor...) count as floor. A Fisher linear discriminant then gives the colour direction that best separates line from floor, and every later frame is converted to a "line-ness" image (line bright, floor dark) before the usual Otsu + line-lock processing. Skipped when `autoColor = false` or when the colours were given with `setTrackColors()`.
2. **Find straight-ahead**. Collects `calibFrames` (default 10) solid line detections, takes their **median** position as the steering zero reference, and records the typical line width.

It returns `true` once both are done. Progress and problems are reported in the callback (`calibPhase`, `calibProgress`, `calibIssue`):

CALIBISSUE	MEANING	WHAT TO DO
ISSUE_NONE	All good	—
ISSUE_LOW_CONTRAST	The colours in view are too similar, or there is only one surface	Make sure the line is in the STEER strip; for same-brightness colours use YUV / ARGB input; even out the lighting
ISSUE_NO_FLOOR	No colour fills both frame edges	Place the robot on the line, with floor visible on both sides
ISSUE_NO_LINE	Colours learned, but no line band found this frame	Check the line is inside the STEER strip

Giving the colours yourself (optional)

```
pipeline.setTrackColors(0xFF000000 /* black line */, 0xFF1E3CC8 /* blue floor */);
pipeline.clearTrackColors(); // go back to learning them
```

Re-calibrating mid-run

```
pipeline.requestRecalibration(); // re-learns colours + straight-ahead
// then feed frames to calibrate() again before resuming processFrame()
```

Recalibrate whenever you move to a track with different colours or very different lighting.

Runtime Utility Methods

```
pipeline.getFramesPerSecond() // frames processed in the last completed second
pipeline.getTotalFrames()    // cumulative frame count since init
pipeline.getElapsedSeconds() // wall-clock seconds since first frame
pipeline.isCalibrated()      // true once calibration is complete
pipeline.isColorModelReady() // true once the line/floor colours are learned (or preset)
pipeline.getLineColorArgb()  // learned line colour, 0xAARRGGBB
pipeline.getFloorColorArgb() // learned floor colour, 0xAARRGGBB
pipeline.requestRecalibration() // re-learn colours + straight-ahead
pipeline.release()           // shut down and reset
pipeline.isInitialized()     // false until the first frame (or after release()/configure())
pipeline.setFrameCallback(cb) // register overlay callback; pass null to unregister
```

Geometry getters (the same values the frame callback delivers — prefer the callback):

```
pipeline.getImgW() / getImgH() // frame size in pixels
pipeline.getHRoiTop() / getHRoiBot() // scan-strip rows
pipeline.getVRoiTop() // far edge row of the look-ahead
pipeline.getVRoiHalfWidthPx() // look-ahead half-width (config)
pipeline.getSteerRefXFrac() / getTrackRefXFrac() // calibrated straight-ahead columns
pipeline.getLookAheadPointCount() / getLookAheadColFracs() / getLookAheadRowFracs()
pipeline.getHorizonYFrac() // floor horizon row from the mount tilt
pipeline.perspectiveWidthScale(yFrac) // relative apparent width at a row
```

Frame Callback — Overlay Geometry

Register a single callback once, and the library calls it automatically on **every calibration frame and every** `processFrame()` **frame**, passing a `FrameData` object with everything needed to draw a visual overlay and a status display: the H-ROI strip, the V-ROI look-ahead band, the dynamic road path, the live centroid, calibration progress and the learned colours.

No extra computation is added. The data already exists inside the pipeline — the callback just packages and delivers it (≈ 0.02 ms per frame). The callback runs on the thread that called `calibrate` / `processFrame`.

```
pipeline.setFrameCallback(data -> {
    // called automatically every frame – use data fields to draw your overlay
});
pipeline.setFrameCallback(null); // unregister
```

Important: the same `FrameData` instance is reused on every call. Copy any fields you need to keep beyond the current frame.

FrameData Fields

FIELD	TYPE	DESCRIPTION
<code>imgW</code> , <code>imgH</code> NEW	<code>int</code>	Frame size in pixels — the image all fractions refer to.
<code>hRoiTopFrac</code>	<code>float</code>	H-ROI top edge — fraction of frame height.
<code>hRoiBotFrac</code>	<code>float</code>	H-ROI bottom edge — fraction of frame height.
<code>vRoiTopFrac</code>	<code>float</code>	V-ROI far (top) edge — fraction of frame height.
<code>ignoreTopFrac</code> NEW	<code>float</code>	Top of the ignored robot-body band — fraction of frame height (1 = nothing ignored).
<code>steerRefXFrac</code>	<code>float</code>	Calibrated straight-ahead column for steering — fraction of frame width.

trackRefXFrac	float	Calibrated straight-ahead column for the look-ahead — fraction of frame width.
halfBotFrac	float	Near (bottom) half-width of the V-ROI box — fraction of frame width.
halfTopFrac	float	Far (top) half-width of the V-ROI box — smaller than halfBotFrac due to perspective taper.
centroidXFrac	float	Detected line position — fraction of frame width. Now the real position (not derived from the clamped steering), also during calibration. -1 if not found.
trackWidthFrac NEW	float	Measured line width — fraction of frame width.
tiltDeg NEW	float	Lean of the look-ahead line from vertical, degrees (positive = leans right).
lookAheadColFrac[]	float[]	Column positions of the dynamic road poly-line, nearest to farthest (size 48).
lookAheadRowFrac[]	float[]	Row positions of the dynamic road poly-line, nearest to farthest (size 48).
lookAheadPointCount	int	Number of valid points in the path arrays.
trackFound	boolean	true when the line was detected in the H-ROI this frame. Use this for "TRACK LOST" — not tracking > 2 (a hard turn also reads 2).
isCalibrated	boolean	true once calibration is complete.
calibPhase NEW	int	CALIB_LEARNING_COLORS, CALIB_FINDING_CENTER or CALIB_DONE.
calibProgress NEW	float	Overall calibration progress, 0 ... 1.
calibIssue NEW	int	Current calibration problem (ISSUE_*, see Calibration).
colorModelReady NEW	boolean	true when frames are processed in learned-colour mode.
lineColorArgb, floorColorArgb NEW	int	Learned colours (0xAARRGGBB) — handy for UI swatches.
colorSeparation NEW	float	How well the two colours separate, in noise standard deviations.
threshold NEW	int	Threshold used in the scan strip this frame; -1 = no contrast (no line).
steering	int	Steering value, -100 ... +100 — same as result[0].
tracking	int	Tracking value, min ... 100 — same as result[1].

Drawing the overlay

Every field is a **fraction**, so convert to screen pixels first:

```
float screenX = data.someXFrac * screenWidth;
float screenY = data.someYFrac * screenHeight;
```

Cropped previews: if the preview is scaled to fill the screen (Android `PreviewView` default `FILL_CENTER`), part of the image is cropped. Map with the same scale, or the overlay will drift: `scale = max(viewW / imgW, viewH / imgH)`, `x = (viewW - imgW*scale)/2 + xFrac*imgW*scale` (same for y). RoboPhoneApp's `OverlayView` does this.

H-ROI strip (a full-width horizontal band):

```
float top    = data.hRoiTopFrac * screenHeight;
float bottom = data.hRoiBotFrac * screenHeight;
canvas.drawRect(0, top, screenWidth, bottom, paint);
```

V-ROI trapezoid (perspective-tapered box — the far edge is narrower, and leans toward `trackRefX`):

```

float centre = data.steerRefXFrac * screenWidth;
float shear = (data.trackRefXFrac - data.steerRefXFrac) * screenWidth;
// Near (wide) bottom corners – sit at the H-ROI top edge
float hTop = data.hRoiTopFrac * screenHeight;
float blX = centre - data.halfBotFrac * screenWidth;
float brX = centre + data.halfBotFrac * screenWidth;
// Far (narrow) top corners – shifted by shear so the box leans with the road
float vTop = data.vRoiTopFrac * screenHeight;
float tlX = centre + shear - data.halfTopFrac * screenWidth;
float trX = centre + shear + data.halfTopFrac * screenWidth;
// Connect: (blX,hTop) → (brX,hTop) → (trX,vTop) → (tlX,vTop) → close

```

Dynamic road band (bends through curves):

```

int n = data.lookAheadPointCount;
if (n >= 2) {
    for (int i = 0; i < n; i++) {
        float x = data.lookAheadColFrac[i] * screenWidth;
        float y = data.lookAheadRowFrac[i] * screenHeight;
        // connect points with a poly-line; i = 0 nearest (bottom), i = n-1 farthest (top)
    }
    // As a band: trace the left edge (x - halfWidth) forward, then the right edge back.
    // halfWidth tapers from halfBotFrac (near) to halfTopFrac (far).
}

```

Centroid indicator (live line position):

```

if (data.centroidXFrac >= 0) {
    float cx = data.centroidXFrac * screenWidth;
    float top = data.vRoiTopFrac * screenHeight;
    float bot = data.hRoiBotFrac * screenHeight;
    // vertical line at cx from top to bot – green if data.trackFound, orange if not
}

```

State readout:

```

if (!data.isCalibrated) {
    status.setText("Calibrating " + Math.round(data.calibProgress * 100) + "%");
    // data.calibIssue explains a stall – see the Calibration table
} else if (!data.trackFound) {
    status.setText("Track lost");
} else {
    status.setText("Tracking S=" + data.steering + " T=" + data.tracking);
}
lineSwatch.setColor(data.lineColorArgb); // learned colours
floorSwatch.setColor(data.floorColorArgb);

```

Full Example

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();

// Optional tuning
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
cfg.autoColor          = true;    // learn line + floor colours (any two colours)
cfg.cameraTiltDeg       = 45f;    // phone mount angle
cfg.bottomIgnoreFrac    = 0.20f;  // hide robot body from scan
cfg.trackMemory         = true;    // lock onto line, ignore reflections
cfg.trackingMinValue    = 2;      // never send a full stop
pipeline.configure(cfg);

// Register overlay callback – fires automatically on every frame
pipeline.setFrameCallback(data -> {
    myOverlay.drawHRoi(data.hRoiTopFrac, data.hRoiBotFrac);
    myOverlay.drawVRoi(data.vRoiTopFrac, data.steerRefXFrac,
                       data.trackRefXFrac, data.halfBotFrac, data.halfTopFrac);
    myOverlay.drawRoadPath(data.lookAheadColFrac, data.lookAheadRowFrac,
                           data.lookAheadPointCount);
    myOverlay.drawCentroid(data.centroidXFrac, data.trackFound);

    robot.setSteeringAngle(data.steering);
    robot.setSpeed(data.tracking);
});

// Phase 1 – Calibration (place robot correctly on the line first)
Yuv f = camera.getYuvFrame();    // packed y / u / v planes
while (!pipeline.calibrateYuv420(f.y, f.u, f.v, 1280, 720)) {
    f = camera.getYuvFrame();
}

// Phase 2 – Normal operation
while (running) {
    f = camera.getYuvFrame();
    pipeline.processFrameYuv420(f.y, f.u, f.v, 1280, 720);
    // all results and overlay data delivered automatically via the callback
}
```

Behaviour Reference

SITUATION	STEERING	TRACKING
Straight line, centred	~0	~100
Line drifting to one side	± small	high
Curve approaching (line leaning)	varies	dropping toward the floor
Approaching a dead end / sharp corner NEW	~0	ramps 100 → 2 with the remaining road
Hard turn	± 100	2–30
Line completely lost	± 100 (exactly)	2 (the floor)
Back on a straight after a turn	returns to 0	~100

Notes

- The pipeline is **not thread-safe**. Call `calibrate / processFrame` (and `requestRecalibration()`) from a single thread only — e.g. have a UI button set a flag that the camera thread checks.
- The returned `int[]` is reused on every call. Copy the values if you need them past the next frame.
- Calibration assumes the robot is correctly placed on the line, with floor visible on both sides. If it is not, call `requestRecalibration()` once it is repositioned, then `calibrate` again.
- A line that jumps sideways by more than the line-lock gate (`trackMemoryGateMult × line width`) in a single frame is treated as leaving the line — this is what makes the lock ignore reflections.
- Very harsh scenes can still defeat colour learning: e.g. a dark floor in deep shadow, with strong texture, where the tape is only ~10 grey-levels darker than the shadowed floor. Even lighting helps.
- Performance: ≈ 0.7 ms per frame in colour mode and ≈ 0.13 ms in grayscale mode at 720×1280 on a desktop CPU; all per-frame buffers are pre-allocated (no garbage per frame).

RoboPhoneApp — Android Reference Integration

`RoboPhoneApp/` runs the library live on the phone camera (CameraX → packed YUV 4:2:0 → `calibrateYuv420 / processFrameYuv420`). Everything on screen comes from the `FrameCallback`:

- **Overlay** — cyan STEER strip with the detected line, magenta look-ahead band that bends along the line, dashed calibrated straight-ahead line, red ignored robot-body band; mapped with the preview's `FILL_CENTER` crop so it lines up with the camera image.
- **HUD** — STEER bar (−100...+100), TRACK bar (min...100 %), status (calibration progress and problems / TRACKING / TRACK LOST), FPS, learned LINE and FLOOR colour swatches, threshold and colour separation.
- **RECALIBRATE** button — re-learns the colours and the straight-ahead reference.

The app builds the library **from source** through a Gradle composite build (`includeBuild("../LineFollowerLibrary")`), so library changes show up in the app on the next build — no jar copying. Library tests: `cd LineFollowerLibrary && gradlew test` (format / colour unit tests plus camera-simulator scenes: shadows, wood grain, turns, line loss, distractor tapes, dead ends).